

GDV. MAP

Curso 2024/2025

Pilar Sancho Thomas (psancho@ucm.es)

Marco Antonio Gómez Martín

Práctica 5: Desarrollo con Tests de Unidad.

1. Introducción

En esta práctica haremos una implementación simple para consola del juego “Conecta 4” para poner en práctica los test de unidad. Para hacerla habrá que desarrollar unas pocas clases de C#; ten en cuenta que *no pretende ser un ejemplo de buen diseño de implementación de este tipo de juegos*.

Necesitarás crear una solución de Visual Studio llamada Conecta4 con dos proyectos, uno con el juego (llamado también Conecta4) y otro con los test de unidad (Tests), de forma similar a lo que se hizo en la primera parte de la práctica.

La práctica se divide en cuatro fases. Al final de cada fase se debe hacer una copia del estado del proyecto pues en la entrega se incluirá el estado de esas implementaciones parciales.

- Fase de TDD pura: te proporcionamos unos test de unidad que debes incorporar al proyecto y tendrás que escribir el código que hace que esos test funcionen.
- Desarrollo de test de unidad: te proporcionamos la implementación (parcial) de algunas de las clases del juego y tendrás que escribir sus test de unidad.
- Desarrollo combinado: en esta fase se termina la implementación del juego creando los test de unidad del código generado.
- Parte opcional: tras terminar la implementación completa, se proponen cambios opcionales a la misma.

2. Conecta 4

El Conecta 4 se juega con un tablero en vertical con 7 columnas en las que entran 6 fichas. Cada jugador juega con un color de ficha distinto (en nuestro caso color amarillo y rojo, aunque se escribirán con una O y una X respectivamente en lugar de usar colores) y en su turno deja caer una de sus fichas por alguna de las columnas que no estén completas. El objetivo del juego es colocar cuatro fichas del mismo color seguidas, ya sea en horizontal, vertical o diagonal.

La aplicación es muy simple: se presenta el tablero y se va pidiendo sucesivamente a cada jugador en qué columna se quiere dejar caer su ficha. Tras cada turno se vuelve a escribir el tablero. El proceso termina cuando alguno de los jugadores gana o el juego termina en tablas. En ese momento se pregunta si se quiere volver a jugar.

A continuación, aparece un ejemplo (parcial) de ejecución.

Juegan amarillas (0)

Indica la columna donde colocar la ficha: 1

A diagram of a rectangular box. The top and side boundaries are represented by dashed lines. The bottom boundary is represented by a thick, solid line. The bottom-left corner of the box is labeled with the number '0'.

Juegan rojas (X)

Indica la columna donde colocar la ficha: 1

Juegan amarillas (0)

[...]

Juegan amarillas (0)

Indica la columna donde colocar la ficha: 4

XXX	0000
-----	------

FIN DE JUEGO. Ganan amarillas

Otra partida? (S/N): s

Juegan amarillas (0)

Indica la columna donde colocar la ficha: 4

```
| | | |
| | | |
| | | |
| | | |
| 0 | |
=====
```

Juegan rojas (X)

Indica la columna donde colocar la ficha: 4

[...]

Indica la columna donde colocar la ficha: 7

```
| | | |
| | | |
| 0 | |
| OX | |
| OX X |
=====
```

Juegan amarillas (O)

Indica la columna donde colocar la ficha: 3

```
| | | |
| 0 | |
| 0 | |
| OX | |
| OX X |
=====
```

FIN DE JUEGO. Ganan amarillas

Otra partida? (S/N): n

3. Implementación a vista de pájaro

Todas las clases de la implementación estarán en un namespace llamado Conecta4 y los test estarán en otro llamado Tests. El juego tendrá cinco clases:

- **TipoCasilla:** en realidad es un enumerado para representar el tipo de celda que puede haber en el tablero (vacía, amarilla y roja). Este mismo enumerado se utilizará durante el juego para saber cuál es el color del jugador que tiene el turno (algo que desde el punto de vista de un buen diseño quizá no sea buena idea).
- **Tablero:** almacena el estado del tablero. Puedes utilizar un array bidimensional de **TipoCasilla**. Tiene métodos para colocar una ficha (tanto en una posición cualquiera como dejando caer la ficha desde arriba). La posición (0, 0) hace referencia a la esquina inferior izquierda.
- **Reglas:** contiene una serie de métodos que definen las reglas del juego. Por ejemplo, uno que devuelve el color del jugador que empieza, otro que indica si dado un tablero un jugador ha ganado, etc.

- GUI: clase con todos los métodos estáticos que se encarga de escribir los mensajes presentados al usuario así como hacerle las preguntas.
- Program: clase con el punto de entrada a la aplicación.

4. Fase 1 : TDD

En esta primera fase el punto de entrada a la aplicación permanece vacío. Se deben incorporar al proyecto de tests los proporcionados junto con el enunciado (TestReglas, TestTablero, TestTableroCompleto) e implementar las clases y métodos necesarios para que todos ellos se ejecuten sin problema.

Aconsejamos hacerlo de forma incremental. Primero incorporar al proyecto TestReglas y añadir todo lo necesario para superar sus tests, después TestTablero y por último TestTableroCompleto.

Los métodos concretos que hay que implementar y su funcionamiento no se describen aquí. Debe inspeccionarse el código de los test para averiguar qué métodos son necesarios y hacerse una idea de lo que se espera de cada uno.

Eso sí, para que la fase 4 tenga sentido, el método TableroCompleto de Tablero (que devuelve cierto si todas las celdas están ocupadas) puedes hacerlo recorriendo el array bidimensional del tablero para ver si alguna de sus celdas aún sigue VACIA.

5. Fase 2 : Desarrollo de test de unidad

Para esta fase te proporcionamos la implementación de dos métodos de la clase Reglas:

- PuedePoner: recibe un tablero y una columna y devuelve si el jugador puede poner en esa columna. El número de columna está entre 0 y 6 (al usuario después se le pedirá el número entre 1 y 7).
- CuantasSeguidas: es un método que ayudará después a saber si hay 4 en raya en el tablero. Recibe una posición (x,y) del tablero y una dirección (en forma de incremento en la X y en la Y) y devuelve cuántas casillas seguidas hay del mismo color empezando desde esa posición. Aunque en un buen diseño el método sería privado (se utilizará desde el método público Gana de la fase siguiente), lo dejaremos público para poder tener test de unidad que lo prueben.

Incorpora los métodos al proyecto y genera test de unidad para comprobar que son correctos.

6. Fase 3 : Implementación completa

En esta última fase hay que terminar la aplicación. Su desarrollo se basa en la existencia de la clase GUI que hace todo el trabajo de interacción con el usuario. Con eso y los métodos que faltan por desarrollar, el Main es simple:

```
public static void Main()
{
    Reglas reglas;

    do
    {
        reglas = new Reglas();
    }
}
```

```

    Tablero tablero = reglas.TableroInicial();
    TipoCasilla turno = reglas.QuienEmpieza();

    turno = reglas.ColorContrario(turno); // El "último en poner" fue el contrario al que empieza

    do
    {
        turno = reglas.ColorContrario(turno);

        GUI.PintaTablero(tablero, turno);
        int col;
        do
        {
            col = GUI.PideColumna();
        } while (!tablero.CaeFicha(col - 1, turno));

    } while (!reglas.Gana(tablero, turno) && !tablero.TableroCompleto());

    if (!tablero.TableroCompleto())
        GUI.EscribeGanador(tablero, turno);
    else
    {
        GUI.PintaTablero(tablero, TipoCasilla.VACIA);
        Console.WriteLine(" ==== TABLAS ====");
    }

} while (GUI.JugarOtra());
}

```

Tendrás que implementar el método Gana de Reglas y sus test de unidad y el resto de métodos para que el código anterior funcione. Para la clase GUI (que no tiene test de unidad), puedes utilizar la que se proporciona junto con este enunciado.

7. Fase 4 (opcional): refactorización y extensiones

Si implementaste el método Tablero.TableroCompleto() en la fase 1 recorriendo todo el tablero y buscando alguna casilla vacía sabrás que la implementación es poco eficiente. Refactoriza la clase para que lleve la cuenta de cuántas fichas se han puesto y que lo único que haga sea devolver cierto si ya se han colocado las 7*6 veces. Si los test siguen pasando la refactorización debería funcionar.

Extiende la práctica para preguntar al inicio del juego el tamaño del tablero con el que se quiere jugar en lugar de que sea fijo de 7 columnas y 6 filas. Este añadido requerirá modificaciones tanto en las clases como en los test de unidad.

8. Entrega

La entrega se realizará con un único .zip que tendrá tres (o cuatro) directorios, uno por fase (llamados Fase1, Fase2, Fase3 y Fase4). Cada directorio tendrá el proyecto completo sin los archivos adicionales que genera Visual Studio. En concreto se necesitan los ficheros de solución, proyecto, y ficheros .cs. NO incluyas el directorio .vs ni el packages. La lista de ficheros debería ser parecida a esta (no se incluyen los test de las fases 2 y 3 ni la parte opcional):

```

Conecta4.sln
Tests
Conecta4\App.config
Conecta4\Conecta4.csproj
Conecta4\GUI.cs
Conecta4\Program.cs
Conecta4\Properties
Conecta4\Reglas.cs
Conecta4\Tablero.cs

```

```
Conecta4\TipoCasilla.cs
Conecta4\Properties\AssemblyInfo.cs
Tests\app.config
Tests\packages.config
Tests\Properties
Tests\TableroCompleto.cs
Tests\TestReglas.cs
Tests\Tests.csproj
Tests\TestTablero.cs
Tests\Properties\AssemblyInfo.cs
```

El .zip entregado no debería ocupar más de 500Kb.

En el .zip también se incluirá un archivo txt en el que aparezcan los nombres de los integrantes y **en el que se indique si se ha realizado alguna parte opcional y quién la ha hecho**. El .zip subido incluirá así mismo el nombre de ambos, separado por un guión (*Nombre1Apellidos1-Nombre2Apellidos2.zip*).